R3.09 - SAÉ3.01

Mot de passe

11 octobre 2026 – B. Colombel

Sommaire

1 Avertissements **1**
1.1 Encodage 1
1.2 Fonctions de hachage 2

2 Stockage de mots de passe **2**

3 Comparaison **3**

1 Avertissements

Le TP sera fait en Python3 !
Pensez à choisir le bon noyau avec Jupyter.

1.1 Encodage

Un caractère est en fait une valeur numérique, mais il existe plusieurs manières de considérer ces nombres. On parle de *l'encodage* d'un caractère.
L'encodage le plus courrant est **UTF8** (surtout pour le web) mais il y a également l'**ANSI**, l'**ASCII** ou le **LATIN1**.
Les méthodes `encode()` et `decode()` des chaines de caractères permettent de spécifier les encodages.
L'instruction `txt.encode("...")` va retourner la chaine avec l'encodage spécifié. En particulier cette chaine est une **expression binaire** qui se repère par un `b` au début de son expression. Certains caractères, non reconnu par ce navigateur, s'afficheront alors avec leur code binaire, exprimé en hexadécimale, que l'on repère par `\x` suivit de deux valeurs hexadécimales.
À l'inverse l'instruction `txt.decode("...")` va retourner une chaine à partir d'une chaine en binaire en suivant l'encodage spécifié. Mais si les caractères ne sont pas reconnus par l'encodage alors cela provoquera une erreur.

```
txt="Il était une fois..."
print(txt)
print(txt.encode("utf-8"))
print(txt.encode("latin1"))
print("\n")

txt="Ben ça alors !".encode("utf-8")
print(txt)
print(type(txt))
print(txt.decode())#La valeur par défaut est celle de jupyter = celle du navigateur
                   #en général UTF8
print(txt.decode("ascii"))
#Cette dernière instruction va  provoquer une erreur
#car le caractère 'ç' encodé en utf-8 par le nombre hexadécimale c3,
#c'est à dire la valeur 195 (caractère \xc3),
#ne se décode pas en en ascii
```

1.2 Fonctions de hachage

Nous utilsierons la bibliothèque `hashlib`

```
import hashlib
```

Cette bibliothèque permet d'utiliser de nombreuse fonction de hachage bien connue :
`sha1()`, `sha224()`, `sha256()`, `sha384()`, `sha512()`, `md5()`, `blake2b()`, `blake2s()`, `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, `shake_128()`, `shake_256()`.

Regardons un exemple.

```
s = 'bonjour  '.encode('utf-8')
s2 = 'bonjour !'.encode('utf-8')
```

Les chaînes de caractères sont bien encodées en binaire.
On peut aussi les retrouver en hexadécimales :

```
s.hex(), s2.hex()
```

Appelons l'une de ces fonction `f_hash`. Alors pour hacher une chaine de caractère `txt`, exprimée en binaire, on peut faire `f_hash(txt).hexdigest()`. La méthode `hexdigest()` permet d'afficher l'empreinte en hexadécimale.

```
hashlib.sha256(s).hexdigest(), hashlib.sha256(s2).hexdigest()
```

Exercice 1. La fonction de hachage `SHA-256` a été utilisée pour calculer le haché d'un mot de 5 caractères :

$$9e69e7e29351ad837503c44a5971edebc9b7e6d8601c89c284b1b59bf37afa80$$

Saurez vous écrire un code qui génère tous les mots de passe composés d'exactement 5 caractères parmi les lettres de l'alphabet en minuscules et construire une table de hachage vous permettant de découvrir immédiatement le mot de passe correspondant au haché ci-dessus ?

2 Stockage de mots de passe

Dans ce TP, on utilise la bibliothèque `sqlite3` pour manipuler les bases de données. Voir la documentation.

```
import sqlite3

# ouvrir la connexion ou créer la base
connection = sqlite3.connect("donnees.db")

# Créer un curseur
curseur = connection.cursor()

# Créer la table
curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs (name TEXT, password TEXT)")
```

Exercice 2. Niveau de sécurité $-\infty$
La fonction suivante permet de créer un utilisateur et de stocker son identifiant et son mot de passe dans la base de données.

```
def AjoutUtilisateur() :
    nom = input("login : ")
    curseur.execute("SELECT name FROM utilisateurs WHERE name == '" + nom + "'")
    res = curseur.fetchall()
    if (nom,) in res :
```

```
        print("Utilisateur déjà inscrit")
    else :
        mot_de_passe = input("Entrer votre mot de passe : ")
        mot_de_passe_bis = input("Entrer à nouveau votre mot de passe : ")
        if mot_de_passe == mot_de_passe_bis :
            curseur.execute("INSERT INTO utilisateurs (name, password) VALUES (?, ?)", (nom, mot_de_passe))
            connection.commit()
        else :
            print("Erreur de saisie")
```

1. Après avoir exécuté au moins une fois `AjoutUtilisateur()`, affichez toutes les données stockées dans la base.
2. Écrire une fonction `Verification()` permettant l'authentification d'un utilisateur en vérifiant si un login et un mot de passe entrés en mode console sont corrects.

Exercice 3. Niveau de sécurité -2
En vous inspirant des fonctions précédentes, créer des fonctions `AjoutUtilisateur_sha256()` et `Verification_sha256()` qui modifie le stockage des mots de passe en les hachant avec `SHA-256`.
Attention au format des objets passés en paramètre et renvoyés par les commandes de cette librairie.

Exercice 4. Niveau de sécurité -1
En vous inspirant des fonctions précédentes, créer des fonctions `AjoutUtilisateur_saltedsha256()` et `Verification_saltedsh` qui modifie le stockage des mots de passe en les hachant avec `SHA-256` et en ajoutant un sel global que vous aurez choisi et écrit en dur dans votre code.

Exercice 5. Niveau de sécurité 0
En vous inspirant des fonctions précédentes, créer des fonctions `AjoutUtilisateur_saltedsha256()` et `Verification_saltedsha256()` qui modifie le stockage des mots de passe en les hachant avec `SHA-256` et en ajoutant un sel aléatoire qui sera stoké dans une autrebase de données.
Vous pourrez utiliser la librairie `random` (méthode `getrandbits(n)` et fonction `hex()` de Python pour générer un sel de 16 octets en hexadécimal (32 caractères hexa).
Voir https://docs.python.org/3/library/random.html

Exercice 6. Niveau de sécurité 1
En créant de nouvelles fonctions à partir des fonctions précédentes, modifier le stockage pour utiliser pour le hachage la librairie `bcrypt` de Python.

3 Comparaison

Exercice 7. Dans cet exercice, nous allons évaluer les temps de vérification pour chacun des 5 types de stockage des mots passe et les comparer.

1. Écrire une fonction `genererermdp(N,n,alph)` permettant de générer N mots de passe aléatoires de n caractères pris dans l'alphabet `alph`.
   Vous pourrez pour cela utiliser la commande `choice` de la bibliothèque `random`. Vous stockerez les logins et les mots de passe en clair dans une table `utilisateurs`.
2. Pour chacun des 5 types de stockage des mots de passe vus dans les exercices précédents, écrire une fonction permettant d'effectuer le stockage de l'ensemble du contenu de la table `utilisateurs` dans une nouvelle table.
3. Pour chacun des 5 types de stockage des mots de passe vus dans les exercices précédents, écrire une fonction d'authentification d'un utilisateur sur la seule donnée de son login, en utilisant la table utilisateurs pour récupérer son mot de passe en clair.
4. Évaluer le temps d'authentification de l'ensemble des utilisateurs, pour une valeur de N assez élevée. À titre d'exemple, le mode de stockage le plus lent prend un temps de vérification de l'ordre de 10 secondes pour 50 utilisateurs sur ma machine.
   Pour évaluer les différents temps de vérification, vous pourrez utiliser la bibliothèque `time` de Python. Voir https://docs.python.org/3/library/time.html.

Bonus : Niveau de sécurité 2 En créant de nouvelles fonctions à partir des fonctions précédentes, modifier le stockage pour chiffrer en plus les hachés des mots de passe avec une clef symétrique AES-256. Vous pourrez pour cela ajouter la ligne `from Crypto.Cipher import AES` au début de votre script.
Vous utiliserez le mode `CTR` et la clé `YELLOW SUBMARINE`.
Enfin, vous stockerez le `nonce` dans un champ supplémentaire de la table.

Voir https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html et
https://pycryptodome.readthedocs.io/en/latest/src/cipher/classic.html#ctr-mode.